

1. Computational Tool**01A**

- a) 在本节中我们重温了欧几里得的**尺规计算机**及对应的**算法**，你还能想到哪些计算及对应的工具？
- b) 其中对应的算法都是以什么形式**描述**的？与C++语言有何相似与不同之处？

2. Computational Tool**01A**

同样是在船沿刻线做记号，为什么曹冲**横**着刻就是算法，而**竖**着刻（刻舟求剑）却不是？
试用算法的概念做出解释。

3. Finiteness**01A**

本节告诉我们**程序未必就是算法**，比如，我们时常遇到的“死循环”就不满足**有穷性**。

- a) 回顾你的编程经历，还记得自己写过的这类程序吗？
- b) 为尽可能避免这类问题，编程及调试过程中你有什么经验？
- c) 能否设计一个算法，来**自动检测**任何程序中是否存在死循环，从而**彻底根除**？

4. Failstone**01A**

至今仍不能证明，Hailstone序列的长度对任何自然数 n 都有限。现考查如下函数：

$$failstone(p, q) = \begin{cases} 1 & (p = q) \\ 1 + failstone(p - q, p) & (p > q) \\ 1 + failstone(2p, q) & (p < q \text{ and } q \text{ is odd}) \\ 1 + failstone(p, q/2) & (p < q \text{ and } q \text{ is even}) \end{cases}$$

- a) 试编写一个程序，对任何自然数 p 和 q ，计算出 $failstone(p, q)$ （假定字宽足够，不致溢出）；
- b) 是否对任何 p 和 q ，都有 $failstone(p, q) < \infty$ ？换言之，你的程序能否称作算法？

5. Geometric Distribution**01A**

- a) 在本节中我们介绍了“几何分布”，你还见过哪些随机过程也符合这种分布？
- b) 一般地，当其中单次尝试成功的概率 $p \neq 1/2$ 时，**期望**的尝试次数应该是多少？试给出一般性公式并记住。

6. Algorithm Performance**01B1**

本节我们以“最小三角形面积”问题为例说明了，即便对于规模接近甚至完全相同的问题实例，有些算法的性能也会差异极大。现在再考查“整数素因子分解”问题的蛮力算法：对任何待分解的整数 n ，依次用2、3、5、7、11、...做整除测试；每当发现一个素因子 d ，便输出 d ，同时将 n 替换为 n/d ；直到 n 等于1。

- a) 当 n 大致为 10^{100} 时，该算法**最坏**情况下需要做多少次整除测试？
- b) **最好**情况呢？

7. Turing Machine**01B1**

- a) 对照01B2节讲义及对应的演示，了解图灵机的**结构组成及工作过程**；
- b) 阅读increase()算法，掌握其**原理**；
- c) 尝试编写一个decrease()算法，实现对二进制数字串（至少2位）的**递减**。

8. Yet Another Turing Machine**01B1**

讲义中介绍的图灵机，转换函数要求读写头每次都**必须**移动，这也是图灵最初设计的版本。

- a) 试验证：如果要求计算终止时读写头**必须**复位，那么读写头的累计移动次数必然是偶数。
 后来有人做了调整：转换函数中的移动项除了L和R，还**允许**是N——读写头可以停在原位不做移动。
- b) 试验证：若有某个转换函数的移动项是N，那么必然需要修改当前单元的内容，或改变读写头的状态。
- c) 经此调整之后，图灵机的时间成本应当如何度量？

9. RAM**01B2**

- a) 对照01B3节讲义及对应的演示，了解RAM模型的**结构组成及工作过程**；
- b) 阅读ceiling()算法，掌握其**原理**；
- c) 实现multiply()算法，实现整数**乘法**，并估计其时间复杂度；
- d) 尝试编写一个floor()算法，实现**向下取整**的除法，并估计其时间复杂度；
- e) 图灵机与RAM之间有哪些**区别**？
- f) 从编程语言的角度看，RAM与C++有哪些**区别**？

10. Small-o**01C1**

01C1节介绍过“大 $\mathcal{O}$ ”和“大 Ω ”记号，请查阅资料自学了解“小 $\mathcal{O}$ ”和“小 ω ”记号的含义及作用。

11. Fundamental Calculations**01C2**

讲义上指出，我们将RAM中所有**基本操作**的时间成本都视作 $\mathcal{O}(1)$ ，比如加法、减法。那么，乘法、除法呢？

12. Polynomial Logarithm**01C2**

复杂度函数如果满足 $T(n) = \mathcal{O}(f(n))$ ，是否必有 $\log(T(n)) = \mathcal{O}(\log(f(n)))$ ？反过来呢？

13. Exponential Complexity**01C3**

本节通过2-Subset这个问题来说明，**指数复杂度**的问题及算法普遍存在。你还能再举出哪些这类例子？

14. Back-Of-Envelope**01D1**

32位、64位最大的无符号整数，表示为10进制是多少位？

15. Back-Of-Envelope**01D1**

- a) 阅读PA-book和网络学堂上的《编程作业指引》，熟悉实验平台的硬件配置、编译选项以及作业规范；
- b) 在你开始着手做每一道PA题之前，参照所设定的时限估计出可行算法的时间复杂度**上界**。

16. Iteration**01D2**

《习题解析》1-32例举了多种典型的迭代、调用模式，试独立地估计它们的渐近复杂度，并与答案对照。

17. Fractional Series**01D3**

级数之于计算复杂度，无非是用来估计计算的**成本**，比如运行**时间**。具体来说，在RAM等计算模型中，这类成本都统一度量为基础操作的**次数**。你应该注意到，我们温习的级数中不少都是**分数形式**（比如调和级数），它们对于度量本身为**整数**的操作次数有什么作用呢？

18. Harmonic Series**01D3**

温习此前在微积分课堂学习的**调和级数**，确认 $\sum_{k=1}^n 1/k = \mathcal{O}(\log n)$ 。

19. Stirling Approximation**01D3**

温习此前在微积分课堂学习的**Stirling近似**，确认 $\log(n!) = \mathcal{O}(n \log n)$ 。

20. Storage Cost Of Recursive Algorithms**01E1 + 01E2**

01E1、01E2节分别基于**减治**、**分治**的策略，给出了两种递归的`sum()`算法，并分析了**时间复杂度**。

所谓**空间复杂度**，是指除去**输入数据**本身后，计算过程所需的**空间量**。

- 上述算法的**空间复杂度**分别是多少？
- 原本朴素的**迭代**算法呢？
- 一般地，**递归**算法的空间复杂度与哪些因素有关？其中最主要的有哪些？
- 同一算法的**时间**、**空间复杂度**之间，有什么联系？

21. reverse()**01E1**

我们看到，本节的`reverse()`算法无非是先做一次`swap`，再递归调用一次`reverse`。

- 试确认：即便这两行代码（这两步操作）颠倒次序，算法的功能依然正确；
- 如此颠倒之后，算法的时间复杂度会否有所差别？如果有，更好了还是更差了？
- 空间复杂度呢？

22. Master Theorem**01E2**

试证明**Master定理**。

23. Akra-Bazzi Theorem**01E2**

试证明**Akra-Bazzi定理**。

24. Greatest Slice**01E3**

在本节中，我们介绍了效率由低到高的**四种**总和最大区间算法。

- 如果不仅是算出最大总和，还需要具体地给出对应区间的（起始）**位置**，则应对算法作何调整？
- 在最大总和同时出现于多个区间的情况下，应如何约定以消除这类**歧义**？
- 针对你所做的约定，算法还需要作何**调整**？
- 调整后算法的**时间**、**空间复杂度**，相对于与原算法是否有所增加？能否保持不变？

25. GS by Divide-And-Conquer**01E3**

基于**分治**策略设计的总和最大区间算法，选用**中点**来划分子任务，从而由主定理可知其复杂度为 $\mathcal{O}(n \log n)$ 。

- 如果改为总是按25%：75%来划分子任务呢？
- 一般地，对于任何常数 $0 < \lambda < 1$ ，如果总是按 λ ： $1 - \lambda$ 来划分呢？

26. Memoization**01F1**

翻出你以前编写过的一两个递归程序，试着借助**记忆化**技巧降低其复杂度。

27. Longest Common Subsequence**01F2**

在本节中我们采用不同策略，分别给出了几种**最长公共子序列**的算法。

- 如果不仅要算出LCS的长度，还要其中各字符在输出序列中的**位置**，则应对原算法作何**调整**？
- 调整后算法的**时间**、**空间复杂度**，相对于与原算法是否有所增加？能否**保持**不变？

28. Top-Down vs. Bottom-Up**01F1 + 01F2**

本节所实现的两种最长公共子序列算法，都是基于子问题之间的同一套递推关系，因此二者在本质上是等价的，其差异仅在于各自的求解方向：如果说记忆化版是**自顶而下**，那么动态规划版就是**自底而上**。

- a) 就最长公共子序列这个问题而言，两种方式相对而言**各有什么**优点？
- b) 对于其他也可以采用这两种方式来求解的问题，你的上述结论是否**依然**成立？

29. Code Reading

- a) 下载本课程提供的**示例代码包**，在Visual Studio中通过dsacpp.sln打开整个解决方案；
- b) **找出**本章所涉及的项目，分别**编译、运行、观察**运行结果，并**细读**对应的代码。

30. Demo

- a) 下载**算法演示包**，并参照网络学堂公告中所介绍的方法，学会使用excel和applet类型的演示；
- b) 找到本章所涉及的演示，对照讲义经常**把玩**，反复**推敲**。